HelixVPN — Master Glossary (every term, acronym, protocol, RFC)

HelixVPN — Master Glossary (every term, acronym, protocol, RFC)

Revision: 2 **Last modified:** 2026-06-26T12:00:00Z **Status:** active — Volume 0 (Spine, meta & governance) nano-detail document **Rev 2:** Added `challenges` (vasic-digital/challenges, §11.4.27) + `containers` (vasic-digital/containers, §11.4.76/.161) submodule entries; added a `HelixError` entry cross-referencing its R7 rename to the canonical `CoreError`. **Authority:** Subordinate to `../SPECIFICATION.md` §11 (the spine glossary, which this document expands). Where this glossary disagrees with the spine on a term's meaning, the spine wins until amended per §11.4.73.

> **Document role.** The single, comprehensive, alphabetical glossary for the whole HelixVPN `final/` specification set: every domain term, acronym, protocol, RFC, crate, service, and product name used across Volumes 0–10. Each entry is a precise 1–3-sentence definition plus the **owning doc** where the term is specified in depth. It expands the 20-row spine glossary ([SPEC §11]) to full coverage. Where a term names a fact not yet proven on hardware (a measured number, an interop result), the dependency is marked `UNVERIFIED` (§11.4.6) and the owning doc carries the gate that resolves it. No term is invented; every entry traces to a source doc or RFC.

Table of contents

How to read an entry

Each entry is **Term — definition. (owning doc / source)**. Owning-doc shorthand mirrors [SPEC §10] and [MASTER_INDEX]:

- **SPEC** = `SPECIFICATION.md`; **00…11** = the topical/WBS overview docs; **99** = source-coverage ledger.
- `v02-…/v03-…/v04-…/v05-…/v06-…/v10-…` = the nano-detail volume documents.
- Citation ids `[04_ARCH §N]`, `[04_P0/P1/P2]`, `[04_UI]`, `[05_YBO]`, `[SYN]` are the source-research ids defined in [SPEC] metadata.
- A trailing `UNVERIFIED` marks a term whose *quantitative* claim awaits a named gate (G1–G6) or benchmark; the **term** is real, the **number** is the gate's to prove.

Numerals & symbols

- **1→N (one user → N networks)** — HelixVPN's headline differentiator (X1): a single user is given policy-scoped access to *many* joined private networks at once, with per-user ACL routing, rather than one VPN per network. (00 §4.2; `v02-data-plane/routing-and-addressing.md`; FR-301)
- **4-layer test coverage** — the §11.4.4(b) floor: pre-build gate + post-build + runtime/on-device + paired §1.1 meta-test mutation, applied to every fix and feature. (10; §11.4.4)
- **4via6** — Tailscale-style scheme that maps an advertised IPv4 LAN into the tenant's IPv6 ULA space, so two networks both exposing e.g. `192.168.1.0/24` never collide; the overlapping RFC1918 range is encoded as a host suffix inside a `/96` derived from the tenant `/48`. The D4 Camp-A answer. (01; `v02-data-plane/routing-and-addressing.md`; `v03-control-plane/svc-ipam.md`; D4; FR-303)
- **8-platform reach** — the portability differentiator (X5): iOS, Android, Linux, macOS, Windows, Web (Console-scoped), HarmonyOS NEXT, Aurora OS — all from one shared Rust core + Flutter UI. (03; `v04-client/*`; HVPN-NFR-600..609)
- **8-criteria MVP DoD** — the Phase-1 Definition-of-Done: self-host-from-zero, enroll connector+client, authorized-reach + denied-unauthorized, auto-escalate-to-MASQUE, policy-edit <1s, revoke <1s, kill-switch+DNS-leak, no-durable-log + all-three-apps-drive-it. (00 §10.2; 07; FR §M)
- **100.64.0.0/10 (CGNAT space)** — the shared-address (carrier-grade NAT) range proposed by the D4 Camp-B sources (GMI/KMI) as a 1:1-per-network alternative to 4via6; surfaced, not chosen for MVP. (D4; 99; `v03-control-plane/svc-ipam.md`)

- **§1.1 (paired mutation)** — the constitution's meta-test discipline: every gate ships a deliberate mutation that must flip the gate RED, proving the gate cannot bluff. (10; §1.1)

A

- **Access (Helix Access)** — the end-user consumer VPN app (Mullvad-style): connect, pick exit/network, toggle obfuscation, kill-switch, split-tunnel. Flutter UI + Rust `helix-core` + per-platform tunnel shim. (00 §3; 03; `v04-client/helix-ui-flutter.md`)
- **ACL (Access Control List)** — HelixVPN's authorization model is a Tailscale-ACL-flavored grammar (`group:X → net:Y:port/proto`) compiled to enforcement artefacts; default-deny, fail-closed. (`v03-control-plane/svc-policy.md`; FR-202)
- **Advertise/route mode** — the `helix-core` operating mode the Connector runs in: it *advertises* served CIDRs and *routes* LAN traffic, as opposed to the Client's capture mode. (00 §3; `v04-client/helix-core-rust.md`; FR-703)
- **AEAD (Authenticated Encryption with Associated Data)** — the cipher class used both by WireGuard's ChaCha20-Poly1305 and by the Shadowsocks-2022 wrap; encrypts and authenticates in one pass. (`v02-data-plane/transport-shadowsocks.md`; FR-009)
- **AllowedIPs** — the WireGuard per-peer routing/cryptokey-routing list; HelixVPN's policy compiler emits a per-device `AllowedIPs` set as one of two enforcement artefacts (the other being the edge verdict map). An empty `AllowedIPs` = default-deny. (01 §8; `v03-control-plane/svc-policy.md`; FR-203)
- **Anycast** — a routing technique announcing one IP from many regions (BGP) so a client reaches the nearest PoP; the D-GW-SELECT option (b) for latency-critical HA fleets. (`v06-deploy/ha-and-multiregion.md`; D-GW-SELECT)
- **Anonymous device token** — an enrollment credential that requires no email/SSO/PII; a tenant mints it so a device enrolls with `users.email = NULL`, `oidc_sub = NULL`. Mullvad-parity row F15. (`v05-security/identity-and-enrollment.md`; `v03-control-plane/svc-identity.md`; FR-102; HVPN-NFR-307)
- **Anti-bluff covenant** — the constitution's §11.4 family of rules: a PASS requires positive captured evidence that the feature works for the end user; metadata/config/absence-of-error/grep-only PASS is forbidden. The governing doctrine of Volume 8. (10; §11.4)
- **ArkTS** — the TypeScript-based application language of HarmonyOS/OpenHarmony; the HarmonyOS shim bridges ArkTS → NAPI → the Rust `.so`. (`v04-client/shim-harmonyos.md`; FR-1009)
- **Aurora OS** — the OMP-Russia Sailfish-derived mobile OS; HelixVPN targets it via the omprussia Flutter fork + a Qt/C++ tun shim + signed

RPM (an enterprise SKU, Russian-hosted toolchain). (09; `v04-client/shim-aurora.md`; FR-1010)

- **Audit events (`audit_events`)** — the durable, append-only table recording **control** actions only (who-did-what to identity/policy/devices) — never traffic/destinations/flows. Append-only enforced by `REVOKE UPDATE,DELETE`. (`v05-security/audit-and-compliance.md`; `v03-control-plane/svc-telemetry.md`; FR-605)
- **Auto-ladder (transport-selection ladder)** — the client's automatic transport escalation on handshake failure: plain UDP → LWO → MASQUE/QUIC → Shadowsocks → UDP-over-TCP, driven by handshake-failure events (Mullvad's exact UX). (01 §5.2; `v02-data-plane/transport-selection-ladder.md`; FR-006)

B

- **Bare-link throughput** — the unobfuscated network's raw throughput, the denominator for Phase-0 gate G1 (plain-UDP WG must reach ⩾80% of it). `UNVERIFIED` until G1. (06; HVPN-NFR-001; G1)
- **BGP (Border Gateway Protocol)** — the inter-domain routing protocol that an Anycast HA topology (D-GW-SELECT option b) depends on. (`v06-deploy/ha-and-multiregion.md`)
- **boringtun** — Cloudflare's userspace WireGuard implementation (Rust); HelixVPN's `helix-wg` wraps it as the cross-platform userspace fallback where a kernel/native WG path is unavailable (iOS, in-process edge). (01; `v02-data-plane/wireguard-core.md`; D-CORE-*)
- **buf** — the Protobuf build/lint/breaking-change tool driving the `helix-proto` → Go/Dart/Rust codegen; `buf generate` plus pinned local `protoc-gen-*` plugins. (`v06-deploy/codegen-pipeline.md`; D-CODEGEN-NET)

C

- **CA (Certificate Authority)** — the per-tenant signing authority for short-lived mTLS device certs; the tenant CA key is the single root secret to protect. MVP topology = a single online issuing CA under an offline/KMS root. (`v05-security/pki-and-certs.md`; `v03-control-plane/svc-pki.md`; FR-108; D-PKI-CA-TIER)
- **`ca_chain`** — the certificate chain (length ⩾ 1) delivered to a device at enroll so clients pin a chain from day one; promoting CA topology grows the chain additively. (`v05-security/pki-and-certs.md`)
- **Capture mode** — the Client's `helix-core` mode: it captures the device's traffic into the tunnel (vs the Connector's advertise/route mode). (`v04-client/helix-core-rust.md`)

- **CarrierHealth** — the `TransportConn::health()` return (rtt, loss, handshakes_failed) the auto-ladder reads to decide escalation. (SPEC §7.1; `v02-data-plane/transport-trait.md`)
- **CGNAT** — see **100.64.0.0/10**.
- **ChaCha20-Poly1305** — WireGuard's AEAD cipher; part of the never-forked crypto core (P2). (01; `v02-data-plane/wireguard-core.md`; FR-001; HVPN-NFR-400)
- **`challenges` (vasic-digital/challenges)** — the anti-bluff Challenge-bank submodule (§11.4.27): a §11.4.169 mandatory test surface whose banks score PASS only on positive captured evidence; HelixVPN registers one Challenge per user-visible feature. (10 §5.5; `v06-deploy/helix-ecosystem-integration.md`; §11.4.27)
- **CIDR** — Classless Inter-Domain Routing notation for an IP prefix; a Connector advertises the CIDRs its LAN exposes (`route.advertised`). (`v03-control-plane/svc-registry.md`; FR-704)
- **Client** — the end-user role: dials in, gets an overlay IP, reaches its authorized subset of joined networks or uses the gateway as a plain privacy exit. Reuses `helix-core` + `helix-ui`. (00 §3; SPEC §3)
- **CloudNativePG (CNPG)** — a Kubernetes Postgres operator (Patroni-class) for in-cluster HA Postgres; the concrete example for D-K8S-PG-HA option (a). (`v06-deploy/kubernetes.md`; D-K8S-PG-HA)
- **Connect-RPC (Connect)** — the gRPC-compatible RPC framework used for the agent control channel (`Coordinator.WatchNetworkMap`) over QUIC/H2; browser-friendly, schema-generated. (02; `v03-control-plane/svc-api.md`; `v03-control-plane/protobuf-spec.md`)
- **CONNECT-IP** — RFC 9484 IP-proxying over HTTP; an advanced MASQUE datapath carrying raw IP packets (no inner WG) — a Phase-2 native option. (01; `v02-data-plane/transport-masque-quic.md`; FR-012; RFC 9484)
- **CONNECT-UDP** — RFC 9298 UDP-proxying over HTTP/3; the mechanism that wraps WireGuard datagrams to look like web traffic — the core of "MASQUE/QUIC mode". (01; `v02-data-plane/transport-masque-quic.md`; FR-004; RFC 9298)
- **Connector (Helix Connector)** — the network-side role/app: runs inside a private network, dials outbound to the gateway, advertises served CIDRs, routes LAN traffic for authorized clients. The in-network half of "two-way". (00 §3; SPEC §3; FR-7xx)
- **Console (Helix Console)** — the admin app: tenants, users, devices, networks, routes, policies, audit, optional billing. Flutter web+desktop, API-client only (**no** `helix-core`). (00 §3; `v04-client/web-console.md`; FR-6xx)
- **`containers` (vasic-digital/containers)** — the mandated container-orchestration submodule (§11.4.76/.161): every containerized HelixVPN workload (test infra, edge images) boots on-demand through its `pkg/boot`/`pkg/compose`/`pkg/health` primitives — rootless Podman only, no

ad-hoc docker/podman commands. (05; `v06-deploy/helix-ecosystem-integration.md`; `v06-deploy/podman-quadlets.md`; §11.4.76; §11.4.161)

- **Control plane** — the Go services holding routing/policy truth (identity/registry/ipam/pki/policy/coordinator/events/telemetry/api/store); strictly separated from the data plane and never in the packet path (P1). (02; `v03-control-plane/*`)
- **Coordinator** — the Go control-plane brain that builds per-agent network maps from an in-memory topology graph and pushes minimal deltas with a p99 < 1s convergence SLO. (02; `v03-control-plane/svc-coordinator.md`; HVPN-NFR-003)
- **`CoreError`** — the unified 7-variant FFI error enum `{NotStarted, AlreadyStarted, Config, Auth, HostFatal, BadFd, Internal}` every FFI verb returns; `anyhow` stays orchestrator-internal and converts at the boundary. (`v04-client/ffi-surface.md`; REFINEMENT_NOTES R7)
- **Cover traffic** — dummy packets DAITA injects so traffic volume/timing leaks nothing to an analyst; applied above WG. (`v02-data-plane/daita.md`; FR-017)
- **Curve25519** — WireGuard's elliptic-curve Diffie-Hellman primitive (Noise IK); part of the never-forked crypto core. (01; FR-001)

D

- **DAITA (Defence Against AI-guided Traffic Analysis)** — Mullvad's traffic-shaping defence: constant packet sizing + cover traffic + timing perturbation applied *above* WireGuard via a maybenot state machine; orthogonal to the obfuscation transport. (01 §5.2; `v02-data-plane/daita.md`; FR-017; F9; D-DAITA-A)
- **Data plane** — the Rust edge + kernel/userspace WireGuard that forwards packets; keeps only counters + ephemeral routing state, no logs; fail-static when the control plane is down. (01; `v02-data-plane/*`; P1/P7)
- **Default-deny** — the policy posture: the empty policy denies all; a compiler error denies rather than opens (fail-closed). (`v03-control-plane/svc-policy.md`; `v05-security/zero-trust-and-default-deny.md`; FR-201)
- **Delta** — the minimal change-set a `NetworkMapEvent` carries for one agent (vs a full Snapshot); the unit of sub-second convergence. (SPEC §7.2; `v03-control-plane/svc-coordinator.md`)
- **DERP-style relay (`helix-relay`)** — a Tailscale-DERP-style relay the data plane falls back to when direct P2P hole-punching fails (Phase-2 NAT traversal). (01; `v02-data-plane/routing-and-addressing.md`; FR-020; X4)
- **Device cert** — the short-lived (⩽24h, auto-renew) tenant-CA-signed mTLS certificate every agent authenticates the control channel with; revocable < 1s. (`v05-security/pki-and-certs.md`; FR-105/107)

- **DLQ (Dead-Letter Queue)** — the destination for poison events after a delivery-count ceiling, via Redis `XAUTOCLAIM`; prevents consumer spin. (`v03-control-plane/svc-events.md`; HVPN-NFR-207)
- **DoD** — see **8-criteria MVP DoD**.
- **DoH (DNS-over-HTTPS)** — DNS on :443; the D-KS-1 decision documents the off-tunnel-DoH split-tunnel residual rather than claiming it closed. (`v05-security/kill-switch-and-dns-leak.md`; D-KS-1)
- **docs_chain** — the `vasic-digital/docs_chain` Go engine that mechanically keeps documents + DB in sync (the §11.4.106 enforcer); HelixVPN's spec exports + workable-items sync ride it. (05; `v06-deploy/helix-ecosystem-integration.md`; §11.4.106)
- **DPI (Deep Packet Inspection)** — the censorship technique HelixVPN's obfuscation transports evade; the netns + nftables rig simulates a DPI UDP block for gate G2. (01; `v02-data-plane/obfuscation-and-dpi.md`; G2; FR-004)
- **DNS-leak protection** — forcing DNS through the tunnel and blocking plaintext off-tunnel :53, so a query never reveals destinations outside the tunnel. (`v05-security/kill-switch-and-dns-leak.md`; FR-503; F13)

E

- **eBPF** — the in-kernel programmable datapath HelixVPN's edge may use (alongside/with nftables) to enforce verdict maps per-peer. (01 §8; `v02-data-plane/routing-and-addressing.md`)
- **Edge (data-plane edge, `helix-edge`)** — the Rust gateway component terminating MASQUE, running the WG fast path, and applying verdict maps; consumes the same `helix-transport` crate as the client (byte-for-byte reuse). (01; `helix_edge`; D5)
- **ELD (EDID-Like Data)** — *not a HelixVPN term*; appears only in the inherited constitution's AV-testing rules and is out of scope for this product. (constitution §11.4.5)
- **Enrollment** — the device-onboarding flow: device generates its WG keypair (private key never leaves), presents an enrollment token, receives a short-lived mTLS cert + overlay IP. (`v05-security/identity-and-enrollment.md`; FR-102/103/104)
- **Endpoint** — the gateway's public address a transport connects to (`Transport::connect(endpoint,…)`); per-node `GatewayInfo.endpoint` is set by the coordinator. (SPEC §7.1; `v03-control-plane/svc-coordinator.md`)

F

- **Fail-closed** — a compiler error / missing rule denies rather than opens; the security posture of the policy compiler. (`v03-control-plane/svc-policy.md`; FR-201)
- **Fail-static** — the availability invariant (P1): existing tunnels keep forwarding when the control plane is down; convergence is a coordination property, connectivity must survive its breach. (SPEC §4; `v06-deploy/ha-and-multiregion.md`; HVPN-NFR-200)
- **FFI (Foreign Function Interface)** — the Dart⍰Rust boundary (`helix_start/stop/status_stream/…`); the UI is a pure function of the FFI status stream. (SPEC §7.3; `v04-client/ffi-surface.md`; FR-018)
- **FIPS-203** — the NIST standard for ML-KEM (post-quantum KEM); HelixVPN's PQ handshake derives a PSK from ML-KEM mixed into classical WG. (`v05-security/post-quantum.md`; FR-1101; RFC/FIPS-203)
- **flutter_rust_bridge (FRB)** — the codegen tool (v2) generating the Dart⍰Rust FFI for the Flutter apps; mirrored to UniFFI for native shims. (03; `v04-client/ffi-surface.md`; D-FFI-*)
- **Full-tunnel** — routing *all* client traffic via the gateway exit (the plain privacy-exit / Mullvad use case), egress IP = the gateway's. (01; FR-013)
- **Fyne** — a Go desktop UI toolkit proposed by source 09_GCT and **rejected** in favour of Flutter (only Flutter reaches all 8 platforms with one codebase). (99 gap G3)

G

- **G1–G6** — the six Phase-0 spike exit gates: G1 plain-UDP throughput ⩾80%, G2 MASQUE survives DPI ⩾50%, G3 iOS NE memory ceiling +⩾30% headroom (make-or-break), G4 Go-vs-Rust edge benchmark (resolves D5), G5 FRB FFI drives the core, G6 push-reconcile converges with zero polling. (SPEC §8.0; 06)
- **Gateway** — the public-VPS rendezvous role running the control plane (Go) + data-plane edge (Rust + kernel WG); accepts dials, never initiates into private networks. (00 §3; SPEC §3)
- **`GatewayInfo`** — the per-node coordinator-set struct (`endpoint`, `MasqueSNI`) delivered in the `MapDelta`; a regional failover emits `gateway.failover` and pushes a `GatewayInfo`-only delta. (`v03-control-plane/svc-coordinator.md`; D-GW-SELECT)
- **geoDNS** — DNS that resolves a name to the nearest region's edge IP by client geo; the recommended D-GW-SELECT option (a) for Phase-2 (no BGP dependency). (`v06-deploy/ha-and-multiregion.md`; D-GW-SELECT)
- **Gin** — the Go HTTP framework serving the REST (apps) + WS/SSE surface; the thin edge in front of the modular monolith. (02; `v03-control-plane/svc-api.md`)

- **GitOps** / **policy-as-code** — expressing policy as a repo that reconciles into the control plane (Phase-2). (`v03-control-plane/svc-policy.md`; FR-208)
- **gRPC** — the agent RPC transport (via Connect) carrying `WatchNetworkMap`. (02; `v03-control-plane/protobuf-spec.md`)

H

- **HarmonyOS NEXT** — Huawei's HarmonyOS (pure, no AOSP); HelixVPN targets it via an OpenHarmony-SIG Flutter fork + a Network Kit VPN ability (the biggest platform risk, real native tunnel-shim work). (09; `v04-client/shim-harmonyos.md`; FR-1009)
- **`helix-core`** / **`helix_core`** — the Rust data-plane core shared by client, connector, and edge: WG control, transport, reconciler, FFI. The D2 decision (Rust over Go). (03; `v04-client/helix-core-rust.md`; D2)
- **`helix_design`** — the decoupled, reusable design-system submodule (`vasic-digital/helix_design`) — the canonical token source + polyglot exporters; the subject of Volume 10 (OpenDesign mandate §11.4.162). (10; `v10-design/*`; D-DESIGN-EXTRACT; D-OD-1)
- **`helix-edge`** / **`helix_edge`** — see **Edge**.
- **`HelixError`** — the *superseded* name for the unified FFI error enum sketched in [SPEC §7.3]; the R7 reconciliation renamed it to **`CoreError`** (see that entry — the canonical 7-variant type every FFI verb returns). `HelixError` survives only as a label in the `SPECIFICATION.md` §7.3 FFI sketch and MUST NOT be used in new code. (SPEC §7.3; `v04-client/ffi-surface.md`; REFINEMENT_NOTES R7)
- **`helix-ffi`** / **`helix_ffi`** — the Rust crate exposing the `flutter_rust_bridge` + UniFFI surface to Dart/native. (03; `v04-client/ffi-surface.md`)
- **`helix-go`** / **`helix_go`** — the Go control-plane modular monolith. (02; `helix_go`)
- **`helix-proto`** / **`helix_proto`** — the Protobuf + OpenAPI schemas from which Dart/Go/Rust clients are generated (schema-first, zero drift, P8). (02; `v03-control-plane/protobuf-spec.md`; `v06-deploy/codegen-pipeline.md`)
- **HelixQA** / **`helix_qa`** — the `HelixDevelopment/HelixQA` anti-bluff autonomous-QA submodule + its test banks; one of the §11.4.169 mandatory test surfaces. (10 §5.6; `v06-deploy/helix-ecosystem-integration.md`; §11.4.27)
- **`helix-relay`** — see **DERP-style relay**.
- **`helix-transport`** / **`helix_transport`** — the Rust crate implementing the `Transport` trait — one obfuscation implementation, three consumers (client, connector, edge). (01 §7.1; `v02-data-plane/transport-trait.md`)

- **`helix-ui`** / **`helix_ui`** — the Flutter/Dart UI core (design system + screens) shared by all three apps via three flavors. (03; `v04-client/helix-ui-flutter.md`)
- **`helixvpnctl`** — the Cobra CLI for self-host: `init / keys / enroll-token / policy / revoke`; one `helixvpnctl init` stands up the gateway. (05; `v06-deploy/helixvpnctl.md`; FR-901/904)
- **Hole punching** — the NAT-traversal technique that establishes a direct P2P session by simultaneous outbound packets; relay fallback engages on failure. (`v02-data-plane/routing-and-addressing.md`; FR-020)
- **`hostNetwork: true`** — the Kubernetes pod setting that lands UDP/443 directly on the node IP (node IP = gateway IP); the edge runs as a `DaemonSet` with it (D-K8S-EDGE-INGRESS option a). (`v06-deploy/kubernetes.md`; D-K8S-EDGE-INGRESS)
- **h3** — the Rust HTTP/3 library (with `quinn`) implementing the MASQUE CONNECT-UDP carrier. (01; `v02-data-plane/transport-masque-quic.md`)
- **Hybrid PQ** — post-quantum that is layered on (never replaces) classical WG: with PQ off the tunnel is still secure; with PQ on an attacker must break both. (`v05-security/post-quantum.md`; FR-1102; HVPN-NFR-407)
- **Hysteria2** — a turnkey QUIC+obfuscation protocol (with the Salamander obfuscator); the D1 Camp-B primary surfaced by the plurality of analyses, kept as a `quinn`-tuning reference rather than the chosen primary. (01; 08; D1)

I

- **IK (Noise IK)** — the Noise-protocol handshake pattern WireGuard uses (initiator knows responder's static key); part of the never-forked crypto core. (01; `v02-data-plane/wireguard-core.md`; FR-001)
- **IPAM (IP Address Management)** — the control-plane service allocating from the tenant overlay pool, tracking host allocations + 4via6 mappings; collision-free under concurrent enrollment. (`v03-control-plane/svc-ipam.md`; FR-307)
- **iperf3** — the throughput measurement tool capturing gate G1/G2 evidence over the netns rig. (06; `v08-testing/test-rig.md`; G1)

J

- **JetStream (NATS JetStream)** — the persistent NATS streaming layer HelixVPN swaps to at scale (Phase-2) from MVP Redis Streams; the D3 Phase-2 transport. (02; `v06-deploy/ha-and-multiregion.md`; D3)
- **JNI (Java Native Interface)** — the Android bridge from the VpnService Java/Kotlin layer to the Rust `.so` (builder/protect/fd handoff). (`v04-client/shim-android.md`; FR-1005)

K

- **KEM (Key Encapsulation Mechanism)** — the public-key primitive class ML-KEM belongs to; the PQ handshake derives a shared secret/PSK from it. (`v05-security/post-quantum.md`; FR-1101)
- **Kill-switch** — core-owned state (driven by `helix-core`'s state machine) that blocks all plaintext egress when the tunnel is down or escalating transports; enforced per-OS via the OS firewall. (`v05-security/kill-switch-and-dns-leak.md`; FR-501/502; F11)
- **KMS (Key Management Service)** — the cloud HSM-class store holding the offline CA root / backup-encryption keys; the MVP CA topology roots in KMS. (`v05-security/pki-and-certs.md`; 99 G1)
- **KMP (Kotlin Multiplatform)** — a client-core alternative proposed by 06_GRK, recorded as a considered-then-not-chosen option (Rust core won on the iOS NE memory ceiling). (99; 03)

L

- **`last_seen_at`** — the single durable presence derivative; coarsened to ⩾5 min so it cannot reconstruct a session timeline (carries no destination). (`v05-security/no-logging-as-code.md`; HVPN-NFR-305)
- **LINDDUN** — a privacy threat-modeling methodology (linkability/identifiability/…); used alongside STRIDE in the threat model. (`v05-security/threat-model.md`)
- **LWO (Lightweight WG Obfuscation)** — a cheap keyed WG-header obfuscation + padding rung to evade naive WG-signature blocks; the second rung on the auto-ladder. (01 §5.2; `v02-data-plane/transport-lwo.md`; FR-005; F6)

M

- **MapResponse** — Tailscale's network-map document; HelixVPN's `NetworkMapEvent` (snapshot + delta) is the MapResponse-style desired-state spine. (SPEC §4; `v03-control-plane/svc-coordinator.md`)
- **MASQUE (Multiplexed Application Substrate over QUIC Encryption)** — the IETF working-group + mechanism (RFC 9298 CONNECT-UDP over HTTP/3) that wraps WireGuard datagrams to look like web traffic — *the* "QUIC mode". Mullvad's QUIC mode IS WG-over-MASQUE, not a separate protocol (the single most important architectural correction). (01; `v02-data-plane/transport-masque-quic.md`; D1; FR-004; RFC 9298)
- **maybenot** — the Rust traffic-shaping state-machine framework that implements DAITA's padding/timing defences. (`v02-data-plane/daita.md`; FR-017)

- **Melos** — the Dart/Flutter monorepo tool managing the `helix-ui` multi-package tree (3 flavors). (03; `v04-client/helix-ui-flutter.md`)
- **ML-KEM (Module-Lattice KEM)** — the NIST FIPS-203 post-quantum KEM HelixVPN uses to derive a PSK mixed into the WG handshake (hybrid, never PQ-only). Exact byte sizes `UNVERIFIED` pending the PQ doc's cited confirmation. (`v05-security/post-quantum.md`; FR-1101; FIPS-203)
- **Modular monolith** — the Go control-plane architecture: many in-process domain modules (identity/registry/ipam/pki/policy/coordinator/events/telemetry) behind one deployable, package-boundary-disciplined (no cross-store import). (02; `v03-control-plane/architecture-and-wiring.md`)
- **mTLS (mutual TLS)** — both sides present certs; HelixVPN authenticates the control channel with short-lived tenant-CA-signed mTLS device certs (control auth shares no key with the WG data channel). (`v05-security/pki-and-certs.md`; FR-105/106)
- **MTU (Maximum Transmission Unit)** — the path packet-size limit; WG MTU 1420 on plain UDP, reduced under MASQUE, so WG-in-transport never fragments. (`v02-data-plane/transport-plain-udp.md`; FR-016; HVPN-NFR-008)
- **Multi-hop** — entry/exit separation via nested WireGuard with per-hop keys (entry and exit can be in different jurisdictions); Phase-2. (`v02-data-plane/multihop.md`; FR-401; F10)
- **Multi-network model** — the two-way reverse-tunnel topology where one user reaches N joined private networks via per-user ACL routing. (00 §4.2; X1)

N

- **NAPI (Native API)** — the OpenHarmony native-binding layer the HarmonyOS shim uses (ArkTS → NAPI → `.so`). (`v04-client/shim-harmonyos.md`)
- **NAT traversal** — establishing connectivity across NATs (STUN-like discovery + hole punching + relay fallback); Phase-2 direct-P2P. (`v02-data-plane/routing-and-addressing.md`; FR-020; X4)
- **NATS / NATS JetStream** — the message system HelixVPN scales the event bus to (Phase-2) from MVP Redis Streams; see **JetStream**, **D3**. (02; D3)
- **NEPacketTunnelProvider (NE)** — Apple's Network Extension packet-tunnel provider class (iOS/macOS); runs the Rust core under a hard memory ceiling — gate G3 (make-or-break, the reason the core is Rust). (`v04-client/shim-apple.md`; FR-1004; G3; HVPN-NFR-500)
- **Network Kit** — HarmonyOS's networking ability set providing the VPN-tunnel capability the HarmonyOS shim drives. (`v04-client/shim-harmonyos.md`)

- **Network map** — the per-agent desired-state document (overlay IP, *policy-filtered* peers, routes, transport policy, DNS, kill-switch posture) pushed over `WatchNetworkMap`. Peers are need-to-know-filtered server-side. (SPEC §7.2; `v03-control-plane/svc-coordinator.md`; FR-207)
- **`NetworkMapEvent`** — the streamed protobuf message carrying either a `Snapshot` or a `Delta` plus a monotonic `version`. (SPEC §7.2; `v03-control-plane/protobuf-spec.md`)
- **nftables** — the Linux packet-filter framework HelixVPN's edge compiles verdict maps into (and the rig uses to simulate a DPI UDP block). (01 §8; `v02-data-plane/routing-and-addressing.md`)
- **Noise** — the cryptographic protocol framework WireGuard's IK handshake is built on. (01; FR-001)
- **No-logging-as-code** — the architectural guarantee that no durable connection/traffic table exists; a CI schema-lint fails the build if one appears, and the lint is asserted against the *deployed* DB (runtime signature, §11.4.108). (`v05-security/no-logging-as-code.md`; FR-801; HVPN-NFR-300/308)

O

- **Obfuscation** — making the encrypted WG datagrams look like something else on the wire; a *pluggable layer beneath* WG (the `Transport` trait), never a crypto fork. (01 §5.2; `v02-data-plane/obfuscation-and-dpi.md`)
- **OIDC (OpenID Connect)** — the identity protocol for human principals (tenant owner/admin/member); coexists with anonymous device tokens. (`v03-control-plane/svc-identity.md`; FR-101)
- **OpenAPI** — the REST contract spec (spec-first, hand-authored) from which Dart/TS app clients generate; handlers validated against it. (`v06-deploy/codegen-pipeline.md`; D-OPENAPI-AUTHORING)
- **OpenDesign** — the mandatory (§11.4.162) design authoring/refinement system; in HelixVPN it is the design *authoring* layer while `helix_design` owns the canonical token source + polyglot exporters (the D-OD-1 reconciliation). (10; `v10-design/opendesign-foundation.md`; D-OD-1)
- **OpenHarmony** — the open-source base of HarmonyOS; its SIG maintains the Flutter fork the HarmonyOS shim builds on. (`v04-client/shim-harmonyos.md`)
- **Overlay IP** — the stable per-node address in the tenant overlay (ULA IPv6 /48 per tenant; advertised IPv4 LANs mapped via 4via6); persists across reconnects. (`v03-control-plane/svc-ipam.md`; FR-304)

P

- **Patroni** — a Postgres HA template (leader election + streaming replication); the HA Postgres story for multi-region (D-K8S-PG-HA option a / CloudNativePG). (`v06-deploy/ha-and-multiregion.md`; `v06-deploy/kubernetes.md`)
- **PDP / PEP (Policy Decision/Enforcement Point)** — the zero-trust split: the control plane *decides* (compiles policy) at compile-time; the edge *enforces* (AllowedIPs + verdict maps) at runtime. (`v05-security/zero-trust-and-default-deny.md`)
- **Podman (rootless)** — the mandated container runtime (§11.4.161); one rootless pod for a homelab, the same images scaling to HA. Docker rootful / sudo forbidden. (05; `v06-deploy/podman-quadlets.md`; FR-902; §11.4.161)
- **Port-hopping** — periodically changing the WG/transport port (and disguising as :443/:53) to evade port-based blocks. (`v02-data-plane/transport-masque-quic.md`; FR-008; F8)
- **Post-quantum (PQ)** — see **Hybrid PQ**, **ML-KEM**, **Rosenpass**.
- **Presence** — live, ephemeral reachability state kept in TTL'd Redis (never a durable table); loss of Redis loses no durable state (fail-static). (`v03-control-plane/svc-events.md`; HVPN-NFR-304)
- **Protobuf (Protocol Buffers)** — the agent-contract schema language (`helix.coordinator.v1`); Dart/Go/Rust clients generate from it. (02; `v03-control-plane/protobuf-spec.md`; P8)
- **PSK (Pre-Shared Key)** — WireGuard's optional symmetric key slot; HelixVPN mixes the ML-KEM-derived secret into it for hybrid PQ. (`v05-security/post-quantum.md`; FR-1101)
- **PWU (Parallel Work Unit)** — a self-contained workable item in the §11.4.58/.93 parallel execution model. (SPEC §11; §11.4.58)

Q

- **quadlet** — a systemd unit format (Podman) declaring rootless containers/pods; HelixVPN ships the gateway as quadlet units (NET_ADMIN, :443/udp, one pod, read-only rootfs). (`v06-deploy/podman-quadlets.md`; FR-906)
- **QUIC** — the UDP-based transport protocol underlying HTTP/3 and MASQUE; HelixVPN's "QUIC mode" is WG-over-MASQUE-over-QUIC, not a bespoke QUIC VPN. (01; `v02-data-plane/transport-masque-quic.md`; D1)
- **quinn** — the Rust QUIC implementation (with `h3`) HelixVPN uses for the MASQUE carrier and as a Hysteria2-tuning reference. (01; `v02-data-plane/transport-masque-quic.md`)

R

- **Reconciler** — the `helix-core` loop that drives the local state to the pushed network map (desired-state, no polling); reconnection state machine lives here. (`v02-data-plane/orchestrator-and-state.md`; FR-015; G6)
- **Redis / Redis Streams** — the MVP event bus + ephemeral KV (TTL presence); consumer groups + `XAUTOCLAIM` DLQ; swapped to NATS at scale (D3). (02; `v03-control-plane/svc-events.md`; D3)
- **Reverse tunnel** — the founding topology: internal hosts dial *outbound* to a public gateway, which relays/routes — no inbound port-forward on any private network. (00; SPEC §1; X2)
- **RFC 2119** — the standard defining MUST/SHOULD/MAY force, used throughout the spec. (SPEC §0)
- **RLS (Row-Level Security)** — PostgreSQL's per-row tenant isolation (`FORCE ROW LEVEL SECURITY`), enforcing multi-tenancy at the database, not only the app layer. (`v03-control-plane/data-model-ddl.md`; FR-110; HVPN-NFR-408)
- **Rosenpass** — a post-quantum WG key-exchange add-on evaluated as an alternative to the ML-KEM-PSK approach; its Kyber-vs-ML-KEM status is `UNVERIFIED` pending the PQ doc's cited confirmation. (`v05-security/post-quantum.md`; FR-1103)
- **Riverpod** — the Flutter state-management library; UI state is a pure function of the FFI status stream + Console WS/SSE folding. (03; `v04-client/state-management.md`)
- **RPM** — the package format for the signed Aurora OS build. (`v04-client/shim-aurora.md`; FR-1010)
- **RTO / RPO (Recovery Time/Point Objective)** — the DR budgets owned by `v06-deploy/disaster-recovery.md` (closing ledger gap G1); NFR-205 names the owner, not a guessed number. (HVPN-NFR-205; 99 G1)
- **Runtime signature** — the §11.4.108 definition-of-done: a fix is done only when its one machine-checkable observable verifies on a *clean* deployment (e.g. schema-lint green against the deployed DB). (10; §11.4.108; HVPN-NFR-308)
- **`runHelixApp(flavor, home, capabilities)`** — the single Flutter entrypoint building all three flavors (Access/Connector/Console) from one tree with per-flavor capability gating. (`v04-client/helix-ui-flutter.md`; FR-1001; HVPN-NFR-600)

S

- **S0–S8** — the Phase-0 spike milestones (the gate-bearing work-breakdown steps). (06)
- **Salamander** — Hysteria2's obfuscation layer (the D1 Camp-B knob set kept as a tuning reference). (01; D1)

- **Shadowsocks (Shadowsocks-2022)** — an AEAD proxy protocol; HelixVPN wraps WG in it as an obfuscation rung for QUIC/UDP-hostile networks (via shadowsocks-rust). (01; `v02-data-plane/transport-shadowsocks.md`; FR-009; F4)
- **Shim (tunnel shim / `TunnelPlatform`)** — the thin per-platform native layer attaching the OS tun device to the Rust core (Apple NE, Android VpnService+JNI, Windows wireguard-nt+service, Linux, HarmonyOS, Aurora; Web = none). (03; `v04-client/shim-*.md`)
- **sing-box** — a Go universal transport-multiplexer framework proposed by 06_GRK and **rejected** (Go conflicts with the Rust-core D2 + iOS memory ceiling; custom Rust `helix-transport` gives byte-for-byte client⍰edge reuse). (99 gap G2)
- **Snapshot** — the full desired-state a `NetworkMapEvent` carries when an agent opens the stream with `known_version=0` (vs a Delta). (SPEC §7.2; `v03-control-plane/svc-coordinator.md`)
- **SPIFFE** — the secure-production-identity framework; HelixVPN's mTLS device cert SAN is a SPIFFE-style ID binding it to a `device_id`. (`v05-security/pki-and-certs.md`)
- **Split horizon** — the default that Connectors can't reach each other and Clients can't reach un-granted networks absent an explicit rule. (`v03-control-plane/svc-policy.md`; FR-206)
- **Split tunneling** — routing some traffic in-tunnel and some out (per-route, and per-app on Android/desktop). (01; FR-014; F12)
- **SSE (Server-Sent Events)** — a live one-way push channel the Console may use (alongside WS) for live state. (`v03-control-plane/svc-api.md`; FR-603)
- **STRIDE** — the threat-modeling methodology (spoofing/tampering/…); paired with LINDDUN in the threat model. (`v05-security/threat-model.md`)
- **STUN** — the NAT-discovery mechanism the P2P path uses to learn external mappings before hole punching. (`v02-data-plane/routing-and-addressing.md`; FR-020)

T

- **Tailscale** — the prior-art coordination model HelixVPN borrows from (network-map / MapResponse, 4via6, ACL grammar); HelixVPN is the self-hosted Mullvad-grade union. (00; SPEC §1)
- **Telemetry** — the counts-only health/audit service (Prometheus metrics, audit_events); carries no flows/destinations; label set ⊆ allow-list (no `tenant_id/device_id/*_ip`). (`v03-control-plane/svc-telemetry.md`; FR-803; HVPN-NFR-306)
- **Terraform** — the IaC tool for the multi-region/DR provisioning + region-failover runbook (Phase-2/DR). (`v06-deploy/disaster-recovery.md`; 99 G1)

- **`Transport` trait** — the single Rust abstraction beneath WireGuard: `connect`/`accept` yield a `TransportConn` (send/recv WG datagrams + health). One implementation, three consumers. (SPEC §7.1; `v02-data-plane/transport-trait.md`; FR-003)
- **`TransportConn`** — a live carrier passing WG datagrams in/out plus the `CarrierHealth` the auto-ladder reads. (SPEC §7.1; `v02-data-plane/transport-trait.md`)
- **`TransportKind`** — the stable transport identifier enum (`PlainUdp | MasqueH3 | Shadowsocks | UdpOverTcp | Lwo`) used by the ladder + counts-only telemetry. (SPEC §7.1)
- **tc netem** — the Linux traffic-control network-emulation tool injecting loss/jitter into the Phase-0 netns rig. (06; `v08-testing/test-rig.md`)
- **tun / TUN device** — the OS virtual network interface the tunnel rides; absent in a browser (the reason Web is Console-only, not a system VPN). (03; `v04-client/web-console.md`; HVPN-NFR-607)
- **`TunnelStatus` / `FfiTunnelStatus`** — the FFI status enum the UI subscribes to (`Disconnected | Connecting(transport) | Connected | Reconnecting | Failed`); `FfiTunnelStatus` is the actual Rust identifier, `ffi::TunnelStatus` the path-qualified name of the same type. (SPEC §7.3; `v04-client/ffi-surface.md`; FR-018; REFINEMENT_NOTES R7)
- **Two-way model** — the gateway stitching the network-side leg (Connector→Gateway) and user-side leg (Client→Gateway) without either side opening an inbound port. (00 §4; FR-306; X2)

U

- **UDP-over-TCP (UoT / udp2tcp)** — the last-resort transport tunnelling UDP inside TCP when UDP is fully blocked. (01; `v02-data-plane/transport-udp-over-tcp.md`; FR-010; F5)
- **ULA (Unique Local Address)** — IPv6 private space (`fc00::/7`); HelixVPN assigns each tenant a ULA /48 as the overlay, into which advertised IPv4 LANs are mapped via 4via6. (`v02-data-plane/routing-and-addressing.md`; `v03-control-plane/svc-ipam.md`; FR-303; D4)
- **UniFFI** — Mozilla's multi-language FFI binding generator; HelixVPN mirrors the `flutter_rust_bridge` surface to UniFFI for the native (Swift/Kotlin) shims. (03; `v04-client/ffi-surface.md`)

V

- **Verdict map** — the nftables/eBPF map compiled from policy that grants/denies per-peer reachability at the edge, keyed (`src_overlay_ip, dst_cidr, l4proto, dport`). The second policy enforcement artefact (with AllowedIPs). (01 §8; `v03-control-plane/svc-policy.md`; FR-203)

- **version (network-map version)** — the monotonic counter on each `NetworkMapEvent`; agents resume a stream by `known_version`. (SPEC §7.2; HVPN-NFR-204)
- **vision_engine** — the `vasic-digital` video-evidence/QA submodule (OCR/vision per §11.4.107/.158) wired for recorded-evidence test verdicts. (10 §5.14; `v06-deploy/helix-ecosystem-integration.md`)
- **VpnService** — Android's VPN API; the Android shim integrates it (builder/protect/fd handoff) with the Rust core via JNI, with background-kill resilience. (`v04-client/shim-android.md`; FR-1005)
- **VMAF** / **SSIM** / **ΔE2000** — full-reference video-quality metrics from the inherited constitution's AV-testing rules; **out of scope** for HelixVPN (no media-playback surface). (constitution §11.4.107)

W

- **WASM MASQUE proxy** — an optional in-page WebAssembly proxy that proxies the *browser's own* traffic to a joined network — never a system VPN (a browser has no TUN). Phase-3, honestly scoped. (09; `v04-client/web-console.md`; FR-1011; HVPN-NFR-607)
- **`WatchNetworkMap`** — the server-streaming RPC every agent opens once: a full Snapshot then a Delta stream of policy-filtered desired state; replaces all polling (p99 < 1s convergence). (SPEC §7.2; `v03-control-plane/svc-coordinator.md`; `v03-control-plane/protobuf-spec.md`; FR-205/207)
- **WFP (Windows Filtering Platform)** — the Windows kernel filter API the Windows shim uses for split-tunnel + kill-switch. (`v04-client/shim-windows.md`; FR-1007)
- **WireGuard (WG)** — the cryptographic core everywhere (Noise IK, Curve25519, ChaCha20-Poly1305); never forked, obfuscation is layered beneath it. (01; `v02-data-plane/wireguard-core.md`; FR-001; P2)
- **wireguard-nt** / **wintun** — the Windows kernel WireGuard / TUN drivers the Windows client integrates behind a privileged service with named-pipe IPC. (`v04-client/shim-windows.md`; FR-1007)
- **WS (WebSocket)** — the bidirectional live channel (with SSE) the Console/clients use for live state. (`v03-control-plane/svc-api.md`; FR-603)

Z

- **Zero-trust** — the security posture: default-deny + need-to-know peer filtering + PDP/PEP split; a node only learns peers a compiled rule grants. (`v05-security/zero-trust-and-default-deny.md`; FR-201/207; HVPN-NFR-402)

RFC index

The protocol RFCs the specification binds to. Numbers cited are from the source research; the *interop* claim each one underpins is the owning doc's gate to prove (marked `UNVERIFIED` where a measured/interop result is involved).

RFC	Title (as used here)	Role in HelixVPN	Owning doc
RFC 2119	Key words for requirement levels (MUST/SHOULD/MAY)	Requirement-force convention across the whole spec	SPEC §0
RFC 9298	Proxying UDP in HTTP (CONNECT-UDP over HTTP/3)	The MASQUE mechanism wrapping WG datagrams — "QUIC mode"	`v02-data-plane/transport-masque-quic.md`
RFC 9297	HTTP Datagrams and the Capsule Protocol	The datagram framing MASQUE CONNECT-UDP rides	`v02-data-plane/transport-masque-quic.md`
RFC 9221	Unreliable Datagram Extension to QUIC	QUIC DATAGRAM frames carrying the WG datagrams (low-overhead path)	`v02-data-plane/transport-masque-quic.md`
RFC 9484	Proxying IP in HTTP (CONNECT-IP)	Advanced native IP-over-HTTP/3 datapath (no inner WG), Phase-2	`v02-data-plane/transport-masque-quic.md`; FR-012
FIPS-203	Module-Lattice-Based KEM (ML-KEM)	The PQ KEM deriving the hybrid PSK	`v05-security/post-quantum.md`; FR-1101

> **`UNVERIFIED` note (§11.4.6).** The RFC *numbers and titles* are as cited in the source research and the owning docs; where the owning doc itself flags an external fact pending the re-run web-research pass (e.g. exact ML-KEM byte sizes, the WireGuard whitepaper PDF), that flag carries here. The deep-research appendix ([`../11-deep-research-appendix.md`](../11-deep-research-appendix.md)) holds the cited (URL + access-date) corpus for 10/10 angles as of Revision 3.

Sources verified

- `../SPECIFICATION.md` §3 (roles), §4 (principles), §5 (architecture + transport layering), §7 (the three cross-cutting contracts: `Transport` trait, `WatchNetworkMap`, FFI), §9 (decision register D1–D8), §11 (the 20-row spine glossary this document expands).
- `../MASTER_INDEX.md` Volumes 0–10 (owning-doc filenames for every term).
- `../v01-product/functional-requirements.md` and `nonfunctional-requirements.md` (FR/NFR ids cross-referenced per term).
- The volume nano-detail docs cited inline: `v02-data-plane/*` (transports, WG, routing, DAITA, ladder), `v03-control-plane/*` (coordinator, events, ipam, pki, policy, telemetry, ddl, protobuf), `v04-client/*` (ffi-surface, helix-core-rust, shims, web-console), `v05-security/*` (threat-model, pki-and-certs, no-logging, kill-switch, post-quantum, audit), `v06-deploy/*` (codegen, podman-quadlets, kubernetes, ha-and-multiregion, helixvpnctl, helix-ecosystem-integration), `v10-design/*` (opendesign-foundation, design-tokens).
- `../99-source-coverage-ledger.md` (the rejected-alternative terms: sing-box G2, Fyne G3, KMP, CGNAT/100.64).

Constitution bindings: §11.4.44 (revision header), §11.4.6 (no-guessing — `UNVERIFIED` on terms whose numeric/interop claim awaits a named gate; no term invented), §11.4.65/.153 (HTML+PDF[+DOCX] exports follow in refinement), §11.4.35 (this glossary is the consumer-side reference; the spine glossary §11 is canonical for the 20 spine terms).

Honesty note (§11.4.6): every entry traces to a cited source doc or RFC; no definition is asserted from memory. Terms inherited from the constitution but out of HelixVPN scope (ELD, VMAF/SSIM/ΔE2000) are marked as such rather than silently omitted (§11.4.118).